Tài liệu kỹ thuật và thiết kế hệ thống

Robot di động đa hướng 4 bánh điều khiển bằng cử chỉ hai bàn tay
Dự án AI-Powered Omni Robot, phiên bản 2

Tai Nguyen Anh

Tháng 11 năm 2026

Mục lục

1 Tổng quan hệ thống **2**
1.1 Bài toán và mục tiêu . . . 2
1.2 Chỉ tiêu và kết quả đo . . . 2
1.3 Kiến trúc tổng thể . . . 2

2 Cơ khí và động học **3**
2.1 Cấu hình . . . 3
2.2 Động học nghịch và bảo toàn hướng . . . 4
2.3 Động học thuận . . . 4

3 Phần cứng và điện **4**
3.1 Nguồn và mạch công suất . . . 4
3.2 Quy hoạch chân GPIO . . . 5

4 Phần mềm **6**
4.1 Nhận diện hai bàn tay . . . 6
4.2 Con trỏ giao diện . . . 7
4.3 Giao thức truyền thông . . . 8
4.4 Điều khiển vòng kín trên ESP32 . . . 8
4.5 Giao diện web . . . 9

5 Kiểm thử và phân tích số liệu **10**
5.1 Kiểm thử tự động . . . 10
5.2 Số liệu chạy thật . . . 11
5.3 Phiên thứ hai: vòng thị giác khi có tay, độ trễ đầu cuối và con trỏ gần mép khung 13

6 Giới hạn và cách tái lập **14**

1 Tổng quan hệ thống

1.1 Bài toán và mục tiêu

Robot dẫn động vi sai không trượt ngang được, còn robot omni bố trí 4 bánh chéo 45° chuyển động theo mọi hướng trên mặt phẳng với ba vận tốc độc lập: tiến (v_x), ngang (v_y) và xoay (ω). Giao diện điều khiển cử chỉ thường chỉ lo việc lái, còn đổi chế độ, chỉnh tốc độ hay bấm dừng khẩn cấp vẫn cần chuột hoặc bàn phím.

Dự án làm hai việc cùng lúc, chỉ với một camera laptop:

- **Tay thuận** điều khiển giao diện web: di con trỏ, giữ yên để click, duỗi ngón giữa để bấm hoặc kéo cần lái và thanh trượt.
- **Tay không thuận** lái robot bằng 8 hướng chỉ tay và chữ V để xoay. Ở chế độ Thủ công (cần lái ảo, bàn phím) chỉ cần một tay bất kỳ để điều khiển giao diện.

Phần cứng thực thi là robot dùng ESP32, hai mạch cầu H L298N và bốn động cơ GM25-370 có encoder. ESP32 tự phát Wi-Fi và nhận lệnh qua UDP.

1.2 Chỉ tiêu và kết quả đo

Các chỉ tiêu trong Bảng 1 do nhóm tự đặt làm mốc tham chiếu, không phải yêu cầu từ bên ngoài. Cột kết quả lấy từ các lần đo trong mục 5.

Bảng 1: Chỉ tiêu đặt ra và kết quả đo được.

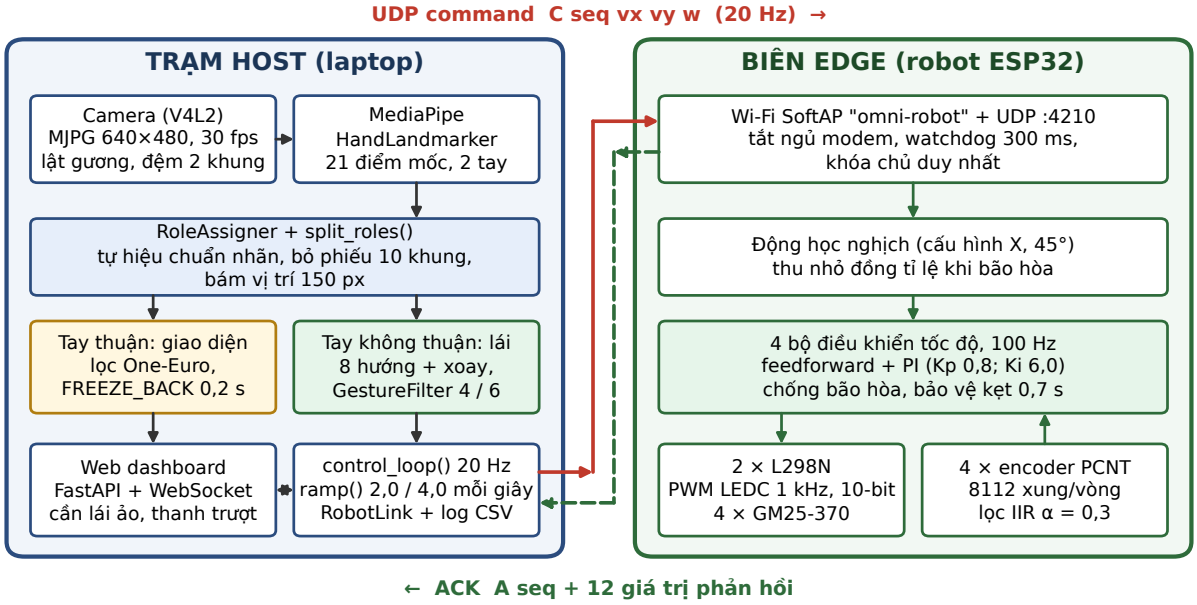
Hạng mục	Chỉ tiêu	Kết quả đo
Tốc độ vòng camera + nhận diện tay	≥ 25 fps	29,9 fps khi không có tay (phép đo riêng); 18,0–25,5 fps trong các phiên server, suy luận 45,5–55,5 ms khi có một tay
Độ trễ khứ hồi UDP (RTT)	< 50 ms (trung vị)	trung vị 6,6–8,1 ms; P95 16,1–26,1 ms
Sai số bám tốc độ từng bánh (trung bình)	$< 5{,}0\,\%$	1,5–3,2 % tốc độ tối đa; P95 4,4–7,4 %
Sai lệch vận tốc thân robot so với lệnh	$< 5{,}0\,\%$	0,1–2,5 % theo trục chính; trục khác $\leq 0{,}004$
Từ lúc chốt cử chỉ đến khi bánh đạt 50 ,		
Ngắt an toàn khi mất lệnh	< 500 ms	ESP32 300 ms; laptop 400 ms; chống kẹt bánh 700 ms

Chú ý: tốc độ 29,9 fps và suy luận 18,5 ms đo ngày 12/11/2026 trên 359 khung liên tiếp; độ trễ cử chỉ đến bánh đã gồm giới hạn gia tốc có chủ ý (mục 4.4) và chưa gồm 4 khung (khoảng 133 ms) cần để chốt cử chỉ.

1.3 Kiến trúc tổng thể

Hệ thống gồm hai phân hệ nối với nhau bằng Wi-Fi do chính ESP32 phát (SSID `omni-robot`, IP cố định `192.168.4.1`, cổng UDP 4210), không cần Internet (Hình 1).

- **Laptop (host)**: `ai/camera.py` đọc ảnh MJPG 1280 × 720 ở 30 fps trong một luồng riêng (nhận diện chạy trên bản sao rộng 640 px), chỉ giữ khung mới nhất. MediaPipe HandLandmarker lấy 21 điểm mốc cho tối đa 2 tay. `RoleAssigner` và `split_roles()` chia tay thuận (con trỏ, `ai/ui/cursor.py`) và tay không thuận (cử chỉ lái, `ai/drive/`). Vòng `control_loop()` 20 Hz chọn lệnh theo chế độ, giới hạn gia tốc rồi gửi UDP qua `RobotLink`.
- **ESP32 (edge)**: nhận gói lệnh, kiểm tra quyền chủ duy nhất và hạn 300 ms, tính động học nghịch, chạy 4 bộ điều khiển tốc độ mỗi 10 ms, xuất PWM ra hai L298N và trả về gói phản hồi gồm tốc độ đặt, tốc độ đo và PWM của 4 bánh.

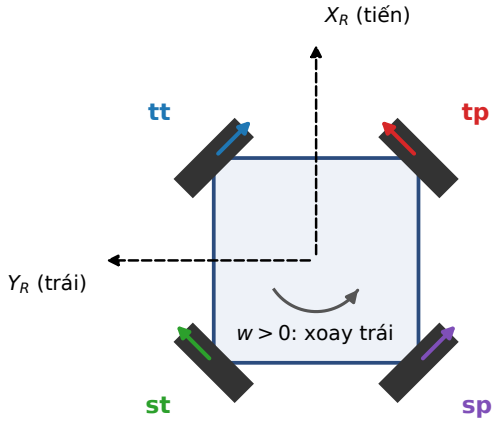


Hình 1: Kiến trúc host–edge. Đường đỏ liền: lệnh UDP; đường xanh đứt: phản hồi kèm số liệu 4 bánh.

2 Cơ khí và động học

2.1 Cấu hình

Bốn bánh omni đặt ở bốn góc, trục lăn chính nghiêng ±45° so với trục dọc thân (kiểu chữ X), gắn trực tiếp vào trục ra của bốn động cơ giảm tốc 1:78. Mã dùng quy ước ROS: $+x$ về phía trước, $+y$ sang trái, $+\omega$ ngược chiều kim đồng hồ khi nhìn từ trên (Hình 2). Chỉ số bánh: 0 trước trái (tt), 1 trước phải (tp), 2 sau trái (st), 3 sau phải (sp). Tốc độ bánh dương là chiều làm robot tiến.



Hình 2: Hình học cấu hình X và hướng đẩy của từng bánh khi quay tiến.

2.2 Động học nghịch và bảo toàn hướng

Trong firmware (`firmware/src/kinematics.h`) vận tốc được chuẩn hóa vào $[-1, 1]$, mức 1,0 ứng với `MAX_CPS` = 4300 xung/giây (khoảng 31,8 vòng/phút). Với $\mathbf{u} = [v_x, v_y, \omega]^\top$:

$$\begin{bmatrix} r'_{tt} \\ r'_{tp} \\ r'_{st} \\ r'_{sp} \end{bmatrix} = \begin{bmatrix} 1 & -1 & -1 \\ 1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & -1 & 1 \end{bmatrix} \mathbf{u}, \qquad r_i = \frac{r'_i}{m}, \quad m = \max\bigl(1, \max_j |r'_j|\bigr). \tag{1}$$

Chia chung cho m giữ nguyên tỉ lệ giữa các bánh khi có bánh vượt 1,0, nên robot chỉ chậm lại chứ không đổi hướng. Nếu cắt từng bánh riêng, tỉ lệ bị méo và robot đi lệch.

2.3 Động học thuận

Các cột của ma trận trong (1) trực giao và có bình phương độ dài bằng 4, nên nghịch đảo giả là ma trận chuyển vị chia 4. Từ tốc độ đo m_i của 4 bánh, `tests/analyze_drive_log.py` tính lại vận tốc thân:

$$\begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}_{\text{đo}} = \frac{1}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix} \begin{bmatrix} m_{tt} \\ m_{tp} \\ m_{st} \\ m_{sp} \end{bmatrix}. \tag{2}$$

3 Phần cứng và điện

3.1 Nguồn và mạch công suất

Nguồn chính 12 V nuôi hai L298N và bốn động cơ. L298N dùng transistor lưỡng cực nên sụt áp nội bộ lớn, cỡ 1,5–2 V theo đặc tính chung của loại IC này (chưa đo riêng trên mạch), vì vậy chọn 12 V. Tốc độ tối đa đo được của bốn bánh là 4552–4869 xung/giây (33,7–36,0 vòng/phút); `MAX_CPS` lấy 4300, thấp hơn bánh chậm nhất để bộ điều khiển còn dư PWM mà bù. PWM dùng ngoại vi LEDC của ESP32 ở 1 kHz, 10 bit, vì thời gian đóng cắt của L298N chậm.

Encoder cấp 3,3 V cho khớp mức logic với ESP32, ngõ ra kiểu đẩy-kéo nên điện trở kéo nội bị tắt trong phần mềm, và bốn encoder dùng chung dây GND với ESP32.

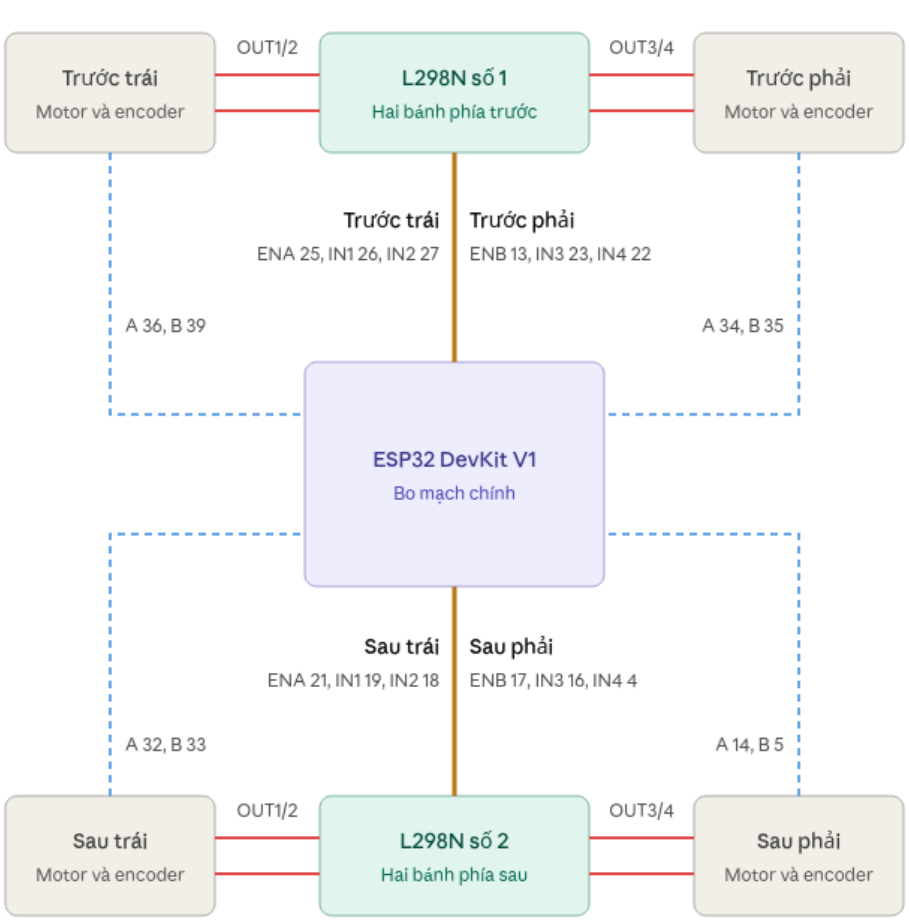
3.2 Quy hoạch chân GPIO

Có 20 chân tín hiệu: 4 PWM, 8 chân chiều (IN1, IN2) và 8 chân encoder (pha A, B). Các chân chỉ-vào của ESP32 (GPIO 34, 35, 36, 39) dùng cho encoder hai bánh trước, nhường chân xuất cho L298N (Bảng 2, sơ đồ ở Hình 3).

Bảng 2: Chân GPIO (`firmware/src/robot_config.h`).

Bánh	PWM	IN1	IN2	Encoder A	Encoder B
0 trước trái (tt)	25	26	27	36	39
1 trước phải (tp)	13	23	22	34	35
2 sau trái (st)	21	19	18	32	33
3 sau phải (sp)	17	16	4	14	5

Hai động cơ bên phải lắp đối xứng gương nên encoder đếm ngược dấu: `ENCODER_INVERT` = {false, true, false, true}. Động cơ sau phải đấu dây ngược chiều, được bù bằng `MOTOR_INVERT` = {false, false, false, true}. Nhờ vậy ở mọi chỗ khác trong mã, PWM dương luôn làm bánh quay tiến và tốc độ đo luôn dương khi tiến.



Hình 3: Sơ đồ đấu nối ESP32 DevKit V1, hai L298N và bốn động cơ GM25-370 có encoder (`docs/pinout.png`).

4 Phần mềm

4.1 Nhận diện hai bàn tay

Nhận diện dùng hình học trên 21 điểm mốc của MediaPipe (chế độ VIDEO, ngưỡng tin cậy 0,6), không huấn luyện thêm.

Ngón duỗi. Một ngón duỗi khi khoảng cách từ đầu ngón đến cổ tay lớn hơn 1,15 lần khoảng cách từ khớp giữa (PIP) đến cổ tay. Tỉ số không có đơn vị nên không phụ thuộc khoảng cách tới camera hay góc xoay của bàn tay. Chỉ ngón trỏ duỗi thì hướng chỉ được tính từ gốc ngón (điểm 5) đến đầu ngón (điểm 8) và chia 8 múi 45°.

Phân vai. Ảnh đã lật gương nhưng MediaPipe gán nhãn Left/Right như thể chưa lật, nên nhãn bị ngược từ góc nhìn người dùng. `RoleAssigner` xử lý như sau:

1. Hai tay: vai theo thứ tự hoành độ cổ tay; sai lệch giữa thứ tự này và nhãn (đã đảo) cập nhật điểm $c_k = 0{,}95\, c_{k-1} \pm 0{,}05$, khi $c_k < -0{,}6$ thì đảo quy ước nhãn.
2. Một tay: bỏ phiếu nhãn trong 10 khung gần nhất.
3. Vừa từ hai tay còn một tay và cổ tay dịch dưới 150 px: giữ vai cũ.
4. Chế độ Thủ công (`drive_active = False`): nhánh lái bị khóa, bất kỳ tay nào cũng là tay giao diện.

Lọc cử chỉ lái. `GestureFilter` cần 4 khung liên tiếp (khoảng 0,13 s) để bắt đầu một lệnh chuyển động và 6 khung (khoảng 0,2 s) để dừng, nên một khung nhiễu thoáng qua không làm robot giật.

4.2 Con trỏ giao diện

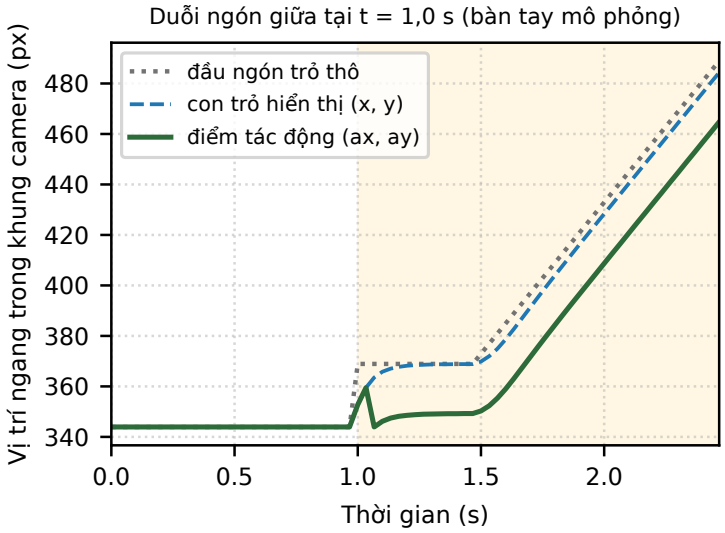
Tay thuận có ba tư thế: *point* (chỉ ngón trỏ) di con trỏ, *click* (trỏ cộng giữa) để bấm hoặc kéo, *idle* (khác) ẩn con trỏ để tránh bấm nhầm. Một tư thế phải giữ 3 khung mới được chấp nhận; mất tay quá 0,3 s thì con trỏ ẩn và mọi nút đang giữ được nhả.

Vị trí đầu ngón trỏ được làm mượt bằng bộ lọc One-Euro ($f_{\min} = 1$ Hz, $\beta = 8$ trên toạ độ chuẩn hóa), tần số cắt $f_c = f_{\min} + \beta|\dot{x}|$ thấp khi tay yên và tăng theo tốc độ tay.

Neo điểm tác động (`FREEZE_BACK`). Duỗi thêm ngón giữa để click có thể kéo đầu ngón trỏ lệch khỏi nút đang nhắm (đo thật ở mục 5). Khi tư thế đổi từ point sang click tại thời điểm t, hệ thống lấy vị trí đã lọc gần nhất cách đó ít nhất 0,2 s làm gốc và giữ độ lệch bù suốt lúc click:

$$(x_a, y_a) = (x_j, y_j),\ j = \arg\max\{t_j :\ t - t_j \geq 0{,}2\}, \qquad (a_x, a_y) = \mathrm{clamp}\big(x + x_a - x(t),\ y + y_a - y(t)\big). \tag{3}$$

Con trỏ hiển thị vẫn bám đầu ngón tay nên kéo vẫn mượt. Sau khi nhả, độ lệch giảm theo hàm mũ với hằng số thời gian 0,15 s (`OFFSET_TAU`). Thanh trượt dùng nắm tương đối: giữ yên để nắm, di ngón tay để kéo, giữ yên lần nữa để nhả. Hình 4 minh họa trên bàn tay mô phỏng.



Hình 4: Neo điểm tác động trên bàn tay mô phỏng (dùng chính lớp `Cursor`). Duỗi ngón giữa tại $t = 1{,}0$ s làm đầu ngón dịch 25 px.

Ánh xạ gần mép khung. Việc nhận diện tay hỏng khi viền khung cắt bàn tay, nên con trỏ trùng đầu ngón không thể tới các góc màn hình. Đặt $d = f - \frac{1}{2}$ là vị trí đầu ngón trong khung ($f \in [0,1]$, theo từng trục), $a = 0{,}10$ là nửa bề rộng vùng giữa và $b = 0{,}44$ là khoảng cách mà tại đó con trỏ phải chạm mép màn hình. Với $k = (0{,}5 - b)/(b - a)^2$ toạ độ đã kéo giãn là $s = \frac{1}{2} + \mathrm{sgn}(d)\min\{\frac{1}{2},\ |d| + k(|d| - a)^2\}$ khi $|d| > a$ và $s = f$ ở các chỗ còn lại, nhờ đó con trỏ bám đầu ngón ở giữa, độ vượt trước tăng dần không có điểm gãy (độ lợi cực đại 1,35) và chạm mép màn hình khi đầu ngón cách viền khung 6 %. Nếu tay mất khi đầu ngón nằm trong 12 % sát viền, con trỏ và tư thế được giữ 1,0 s thay vì 0,3 s. Bản đầu, với đoạn dốc tuyến tính bắt đầu ở $|d| = 0{,}30$ và độ lợi cực đại 1,67, làm con trỏ chạy trước đầu ngón thấy rõ nên đã được thay bằng bản này. Chín trong 55 kiểm tra tự động bao phủ ánh xạ này.

4.3 Giao thức truyền thông

Lệnh chuyển động liên tục 20 Hz chỉ có giá trị mới nhất là hữu ích, nên chọn UDP: gói rớt bị bỏ qua và 50 ms sau có gói thay thế, còn TCP sẽ giữ các gói mới sau gói bị mất. Trên ESP32 tắt chế độ ngủ của modem Wi-Fi (`WiFi.setSleep(false)`) để giảm trễ. Khung truyền là văn bản ASCII, dễ gỡ lỗi:

`C <seq> <vx> <vy> <w>` (laptop → ESP32), ví dụ `C 142 0.500 -0.200 0.000`
`A <seq> <m0 d0 p0 ... m3 d3 p3>` (ESP32 → laptop; muốn, đo, PWM của 4 bánh)

An toàn nhiều tầng. (1) ESP32 chỉ nhận lệnh từ máy gửi hợp lệ đầu tiên và từ chối địa chỉ khác cho đến khi máy đó im quá 300 ms. (2) ESP32 trả lệnh $(0, 0, 0)$ nếu quá 300 ms không có gói mới. (3) Laptop đặt vận tốc đích về 0 nếu quá 0,4 s không có lệnh mới từ chế độ đang chọn. (4) Bộ giới hạn `ramp()` có gia tốc tối đa 2,0/giây, giảm tốc tối đa 4,0/giây, và bắt buộc đi qua 0 trước khi đổi dấu vận tốc.

4.4 Điều khiển vòng kín trên ESP32

Mỗi chu kỳ 10 ms (100 Hz), `firmware/src/wheel.h` đọc số xung từ bộ đếm phần cứng PCNT ở chế độ giải mã vuông pha ×4 (8112 xung/vòng bánh = 26 xung/kênh ×4 × 78) kèm bộ lọc nhiễu phần cứng 12,5 μs, nên việc đếm xung không tốn thời gian CPU. Vì một xung trong 10 ms đã bằng 2,3 % tốc độ tối đa, tốc độ đo được lọc thông thấp bậc 1:

$$\omega_i(k) = 0{,}7\,\omega_i(k-1) + 0{,}3\,\frac{c_i(k)}{4300}, \tag{4}$$

với c_i là số xung trên giây. Mục tiêu r_i của mỗi bánh tiến dần đến giá trị đặt với độ dốc tối đa 4,0/giây. Luật điều khiển (Hình 5):

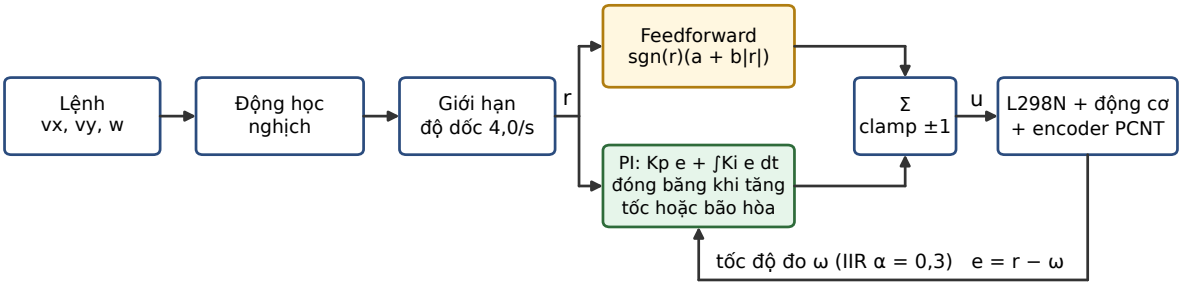
$$u_i^{\mathrm{ff}} = \mathrm{sgn}(r_i)\big(a_i + b_i|r_i|\big), \quad e_i = r_i - \omega_i, \tag{5}$$

$$I_i \leftarrow I_i + K_i e_i\,\Delta t \quad \text{chỉ khi không đang tăng tốc và chưa bão hòa,} \tag{6}$$

$$u_i = \mathrm{clamp}\big(u_i^{\mathrm{ff}} + K_p e_i + I_i,\, -1,\, 1\big), \qquad K_p = 0{,}8,\ K_i = 6{,}0. \tag{7}$$

Hệ L298N cộng GM25-370 rất phi tuyến: ngưỡng PWM để bánh bắt đầu quay khác nhau giữa các bánh (5,5 % đến 14,8 %, gấp 2,7 lần) và khoảng 44 % PWM đã cho 80 % tốc độ không tải. Các hệ số a_i, b_i (Bảng 3) là đường thẳng khớp bình phương tối thiểu từ các điểm xác lập đo khi kê bánh lơ lửng, bằng công cụ `firmware/tools/motor_test.cpp`; độ lệch khớp tối đa 0,045 PWM. Khâu feedforward đưa PWM gần đúng ngay, khâu PI chỉ phải bù phần còn lại (đặc biệt là tải thật trên sàn). Đóng băng tích phân khi đang tăng tốc hoặc bão hòa nhằm tránh tích lũy sai số trong lúc mục tiêu còn chạy (windup). Chưa có phép so sánh với PID thuần hay vòng hở trên cùng quỹ đạo nên chưa định lượng được mức cải thiện.

Bảo vệ kẹt. Nếu PWM từ 0,95 trở lên mà tốc độ đo dưới 0,05 liên tục quá 0,7 s (`STALL_SECONDS`), bánh đó bị ngắt và báo lỗi qua Serial cho đến khi lệnh về 0.



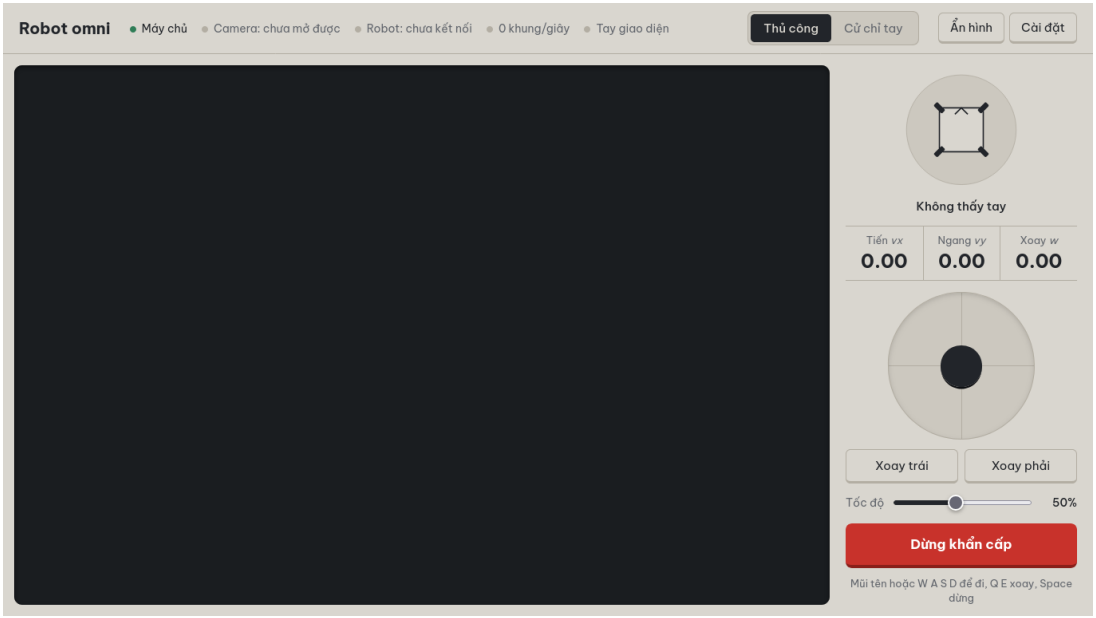
Hình 5: Bộ điều khiển tốc độ của một bánh (chu kỳ 100 Hz).

Bảng 3: Tham số feedforward đo được và sai số bám trung bình trên sàn (% tốc độ tối đa).

Bánh	a_i (`FF_OFFSET`)	b_i (`FF_SLOPE`)	Thủ công v1	Cử chỉ v1	Hai tay v2
tt	0,076	0,470	1,8	1,9	2,5
tp	0,148	0,594	3,2	2,5	2,6
st	0,055	0,453	1,7	1,6	2,3
sp	0,060	0,464	1,5	1,7	2,5

4.5 Giao diện web

Dashboard (FastAPI, WebSocket, một trang không cuộn) có hai chế độ: Thủ công (cần lái ảo, nút xoay, phím mũi tên hoặc W A S D, Q E, Space) và Cử chỉ tay. Mọi nút, thanh trượt và cần lái đều thao tác được bằng con trỏ tay thuận; ánh xạ cử chỉ ở Bảng 4. Hình 6 chụp từ bản chạy thật, không gắn camera.



Hình 6: Dashboard ở chế độ Thủ công.

Bảng 4: Ánh xạ cử chỉ ($s \in [0{,}1; 1]$ là hệ số tốc độ).

Tay	Cử chỉ	Tác dụng
Không thuận	nắm, xòe, mất tay	Dừng: $(0, 0, 0)$
Không thuận	trỏ lên / xuống	Tiến $(+s, 0, 0)$ / lùi $(-s, 0, 0)$
Không thuận	trỏ trái / phải	Sang trái $(0, +s, 0)$ / phải $(0, -s, 0)$
Không thuận	trỏ chéo	Đi chéo $(\pm 0{,}707s, \pm 0{,}707s, 0)$
Không thuận	chữ V trái / phải	Xoay trái $(0, 0, +s)$ / phải $(0, 0, -s)$
Thuận	trỏ	Di con trỏ; giữ yên 0,8 s để click hoặc nắm thanh trượt
Thuận	trỏ cộng giữa	Click nhanh; giữ và di để kéo
Thuận	khác, mất tay	Ẩn con trỏ, nhả mọi nút giữ

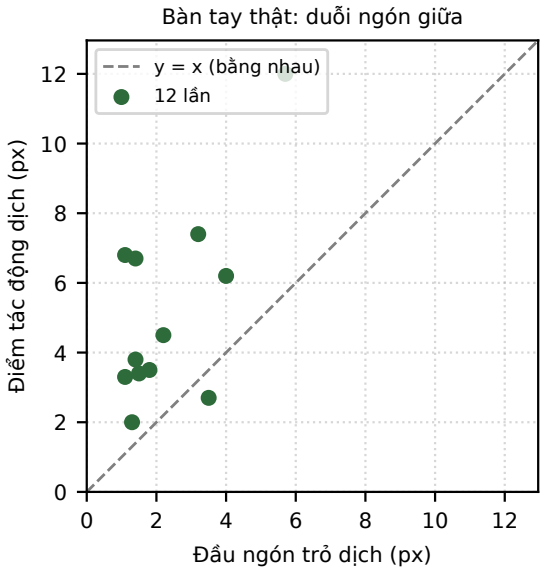
5 Kiểm thử và phân tích số liệu

5.1 Kiểm thử tự động

- `tests/test_hands_logic.py`: 55 kiểm tra trên bàn tay giả dựng bằng toán, gồm phân loại cử chỉ, tư thế giao diện, phân vai khi nhãn bị ngược, tự hiệu chuẩn, bộ lọc One-Euro, neo điểm tác động. Kết quả: 55/55 đạt.
- `tests/handcursor_test.html`: 49 kiểm tra trong trình duyệt cho dwell click, click hai ngón, nắm tương đối thanh trượt, kéo cần lái, nút giữ, tự nhả khi mất tay, thẻ thông số. Chạy bằng Chrome không giao diện (cửa sổ 1280×900) ngày 12/11/2026: 49/49 đạt. Cửa sổ quá thấp (dưới khoảng 500 px) làm một kiểm tra vị trí con trỏ báo lỗi giả vì con trỏ bị kẹp vào mép cửa sổ.

Các kiểm tra tự động dùng bàn tay mô phỏng, nên con số của chúng chỉ mô tả thuật toán trên dữ liệu lý tưởng. Với nhiễu 2 px (200 lần chạy có hạt giống) độ lệch chuẩn của con trỏ giảm từ 0,0047 xuống 0,0016 (giảm 65,5 %). Với ngón trỏ bị lệch $(25, -15)$ px khi duỗi ngón giữa, điểm tác động lệch 0,0126 (đơn vị chuẩn hóa) so với 0,0770 của đầu ngón, giảm 83,6 %; phần dư là 4,6 px trong 29,2 px. Vì tư thế click chỉ được chấp nhận sau 3 khung, điểm tác động có thể lệch thoáng qua tới 15,6 px (khoảng 0,1 s) trước khi bù.

Đo trên bàn tay thật. Chạy `tests/real_hand_eval.py` (cùng lớp `Cursor`, ảnh camera thật, một người thử), dữ liệu `docs/data/v2/real_hand_click_still.csv` và `real_hand_jitter.json`. *Rung khi giữ yên:* giữ yên ngón trỏ 8 giây (209 khung), độ lệch chuẩn của đầu ngón thô là 2,26 px và của con trỏ sau lọc là 2,14 px, chỉ giảm 5 % (mô phỏng cho 65,5 %). Mô phỏng dùng nhiễu độc lập, còn rung và trôi điểm mốc thật nhiều khả năng chậm và có tương quan nên bộ lọc thông thấp đang chỉnh cho nhạy không loại bỏ được; chưa kiểm tra giả thuyết này. *Lệch khi click:* người thử chỉ ngón trỏ vào một điểm và giữ yên (script chờ đầu ngón nằm trong 3 px suốt 1 giây và loại lần nào tay còn trôi), rồi duỗi ngón giữa, 12 lần. Đầu ngón trỏ dịch trung vị 1,65 px (1,1–5,7), nhỏ hơn rất nhiều so với 29,2 px giả định trong mô phỏng. Điểm tác động có neo dịch trung vị 4,15 px (2,0–12,0), nhiều hơn đầu ngón ở 11 trong 12 lần (Hình 7); con trỏ hiển thị dịch 1,9 px. Như vậy với người thử này, khi giữ yên ngón tay, độ lùi 0,2 s làm thêm khoảng 2,5 px thay vì bù độ lệch. Nguyên nhân có thể là độ dịch thật cùng cỡ với rung tay nên neo lấy mẫu đúng phần rung thay vì sửa một cú giật có hệ thống. Một lần đo trước đó không giữ yên ngón tay cho độ dịch trung vị 6,8 px và không được dùng. Đây là 12 lần của một người; chưa đo tỉ lệ bấm trúng, và bán kính dwell (30 px) cùng kích thước nút đều lớn hơn các độ lệch này nên tác động thực tế lên việc bấm nhiều khả năng nhỏ nhưng chưa kiểm chứng. Với người dùng hoặc dáng tay khác mà ngón trỏ giật mạnh hơn, neo vẫn có thể có ích.



Hình 7: Bàn tay thật, 12 lần giữ yên ngón tay: độ dịch của điểm tác động so với đầu ngón trỏ thô khi duỗi ngón giữa. Điểm trên đường đứt là điểm tác động dịch nhiều hơn đầu ngón.

5.2 Số liệu chạy thật

Ba phiên chạy trên sàn nhà, ghi 20 Hz, được phân tích bằng `tests/analyze_drive_log.py`; dữ liệu ở `docs/data/v1/` và `docs/data/v2/` (Bảng 5). Các dòng ghi lúc nguồn motor tắt (53 dòng ở phiên thủ công) được loại bỏ.

Bảng 5: Các phiên chạy.

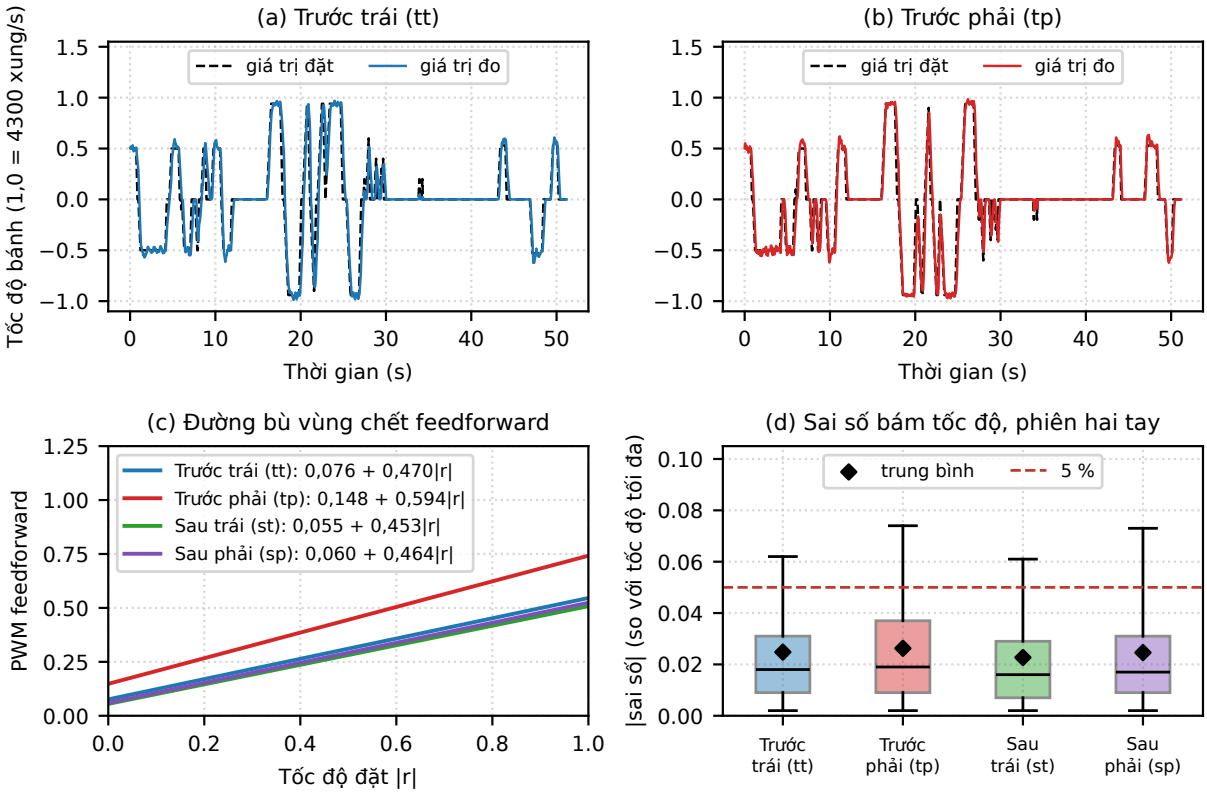
Phiên (tệp)	Dòng	Thời lượng (s)	RTT trung vị / P95 / lớn nhất (ms)
Thủ công v1 (`drive_manual.csv`)	3240	1512	7,8 / 22,5 / 97
Cử chỉ v1 (`drive_gesture.csv`)	4820	247	8,1 / 26,1 / 101
Hai tay v2 (`drive_twohand.csv`)	5020	266	6,6 / 16,1 / 909
Tổng	13080	2026	–

Trong phiên thủ công có 389 dòng ở chế độ cử chỉ. RTT lớn nhất ở phiên v2 là 909 ms, vượt xa hạn 300 ms của ESP32 nên robot sẽ dừng trong trường hợp đó.

Độ trễ. Từ lúc cử chỉ di chuyển được ghi nhận đến khi bánh tải nhiều nhất đạt 50 % mục tiêu mới: trung vị 260 ms (204–359 ms, 38 lần, v1) và 257 ms (203–309 ms, 26 lần, v2). Con số này gồm giới hạn gia tốc có chủ ý nên không phải độ trễ nhỏ nhất có thể.

Độ ổn định cử chỉ. Trong 120 lần đổi cử chỉ (v1) có 7 đoạn ngắn hơn 0,3 s (5,8 %); trong 76 lần (v2) có 5 đoạn (6,6 %). Chưa gán nhãn xem các đoạn ngắn này có phải thao tác ngoài ý muốn.

Bám tốc độ. Chỉ tính khi lệnh không đổi 0,3 s và có độ lớn từ 0,1. Sai số trung bình 1,5–3,2 % (Bảng 3, Hình 8). Phân vị 95 là 4,4–4,8 % ở phiên cử chỉ v1 nhưng 6,1–7,4 % ở phiên hai tay v2, nên chỉ tiêu 5 % đạt theo trung bình chứ chưa đạt theo phân vị 95 ở phiên v2. PWM trung bình của bánh tp cao nhất (0,48–0,49 so với 0,36–0,44 ở các bánh khác), đúng với vùng chết lớn nhất của nó.

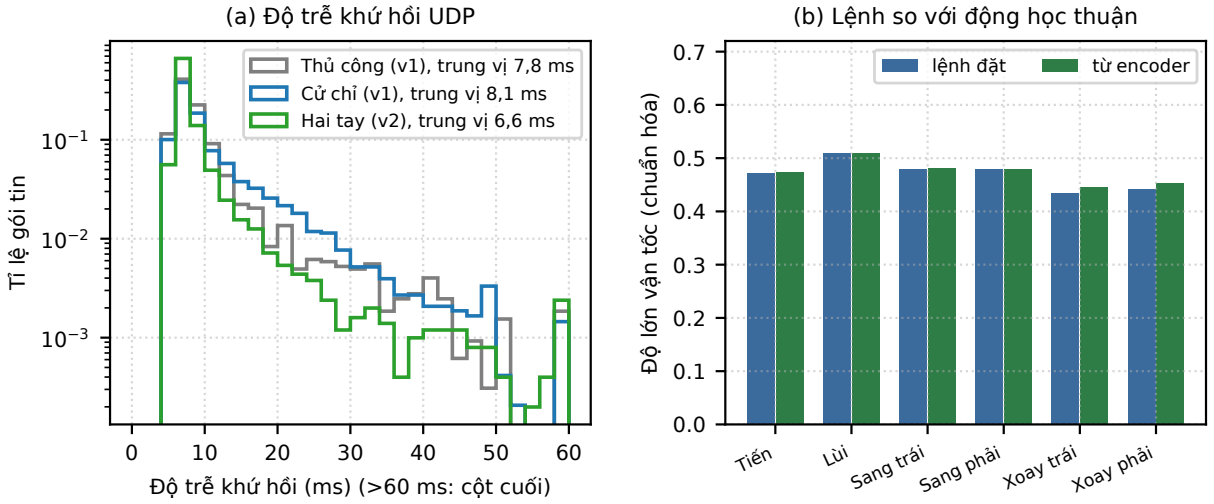


Hình 8: Điều khiển tốc độ bánh, phiên hai tay. (a), (b): giá trị đặt và đo của hai bánh trong 50 s; (c): đường feedforward; (d): phân bố sai số xác lập (râu: phân vị 5–95; hình thoi: trung bình; nét đứt: 5 %).

Chuyển động thân robot. Bảng 6 so lệnh với vận tốc tính từ encoder theo (2); Hình 9 vẽ phân bố RTT và độ lớn vận tốc. Sai lệch theo trục chính dưới 0,6 % khi tịnh tiến và 2,4–2,5 % khi xoay, chuyển động ngoài ý muốn ở các trục khác không quá 0,004. Đây là vận tốc suy ra từ encoder, chưa tính trượt bánh trên sàn nên không phải sai số vị trí thật.

Bảng 6: Lệnh so với động học thuận. Sáu hàng đầu từ phiên cử chỉ v1, hai hàng chéo từ phiên v2.

Chuyển động	Mẫu	Lệnh (v_x, v_y, ω)	Từ encoder	Lệch chính	Trục khác
Tiến	543	(+0,472; 0; 0)	(+0,474; +0,001; +0,002)	+0,6 %	0,002
Lùi	360	(-0,510; 0; 0)	(-0,510; -0,001; -0,001)	-0,1 %	0,001
Sang trái	96	(0; +0,480; 0)	(+0,004; +0,482; +0,002)	+0,4 %	0,004
Sang phải	73	(0; -0,480; 0)	(-0,003; -0,479; 0)	-0,2 %	0,003
Xoay trái	238	(0; 0; +0,435)	(+0,003; +0,001; +0,446)	+2,5 %	0,003
Xoay phải	230	(0; 0; -0,442)	(-0,001; 0; -0,453)	+2,4 %	0,001
Chéo tiến trái (v2)	98	(+0,308; +0,245; 0)	(+0,316; +0,256; +0,001)	–	–
Chéo tiến phải (v2)	71	(+0,313; -0,237; 0)	(+0,316; -0,243; -0,002)	–	–



Hình 9: (a) Phân bố RTT của ba phiên (thang log, giá trị trên 60 ms gom vào cột cuối). (b) Độ lớn vận tốc lệnh và vận tốc từ encoder cho sáu chuyển động cơ bản.

5.3 Phiên thứ hai: vòng thị giác khi có tay, độ trễ đầu cuối và con trỏ gần mép khung

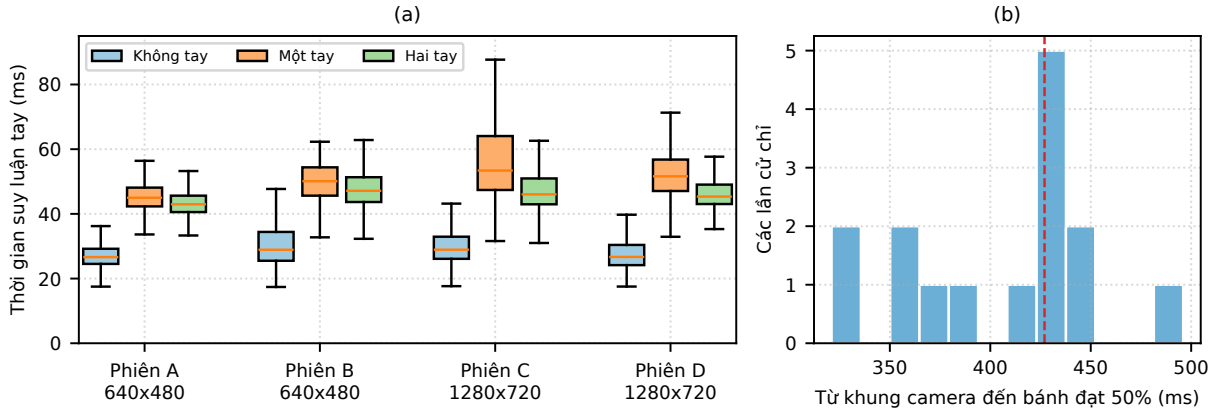
Sau các lần chạy trên, chúng tôi ghi thêm bốn phiên server bằng bộ ghi log từng lần chạy (`web/runlog.py`): mỗi khung camera một dòng, cộng bản ghi 20 Hz của bốn bánh khi robot được nối (dữ liệu ở `docs/data/v2/session2/`, bảng 7). Phiên A và B chạy mã trước các thay đổi mô tả dưới đây (thu hình MJPG 640×480, con trỏ ánh xạ 1:1 theo đầu ngón tay); phiên C và D chạy sau khi nâng thu hình lên 1280×720 (nhận diện tay vẫn trên bản 640 px) và sau khi con trỏ được cho chạy nhanh hơn đầu ngón gần mép khung (C ánh xạ tuyến tính từng đoạn, D ánh xạ mượt ở mục 4.2). A và B có robot nối; C và D chỉ thử con trỏ. A là lần chạy đầu tiên sau khi sửa lỗi một bánh, và D vẫn đang chạy lúc chép log nên D là một bản chụp. Một người vận hành tham gia cả bốn phiên, các phiên khác nhau về việc người đó làm, số tab trình duyệt mở và độ dài, nên đây không phải một so sánh có kiểm soát.

Bảng 7: Các phiên server ghi bằng bộ ghi log từng lần chạy. Thời gian suy luận điểm mốc trung bình (ms) tách theo số tay trong khung; fps là số khung AI mỗi giây trong cả phiên.

Phiên	Camera	Con trỏ	Thời gian (s)	fps	0 tay	1 tay	2 tay
A	640×480	1:1	186	25,5	27,7	45,5	43,3
B	640×480	1:1	76	21,9	31,3	50,1[a]	47,5
C	1280×720	gãy khúc	363	18,0	30,7	55,5	48,3
D	1280×720	mượt	214	21,9	28,9	52,4	47,2

[a]Chỉ có 46 khung có đúng một tay.

Vòng thị giác khi có tay. Khi chưa có tay trong khung, suy luận trung bình 27,7–31,3 ms; một tay 45,5–55,5 ms, hai tay 43,3–48,3 ms (Hình 10a). Tốc độ khung cả phiên là 18,0–25,5 khung/giây, dưới mục tiêu 25 fps và dưới 29,9 fps của phép đo riêng lẻ ở trên (chỉ đo thu hình và nhận diện, không có tay); server còn mã hóa và gửi mọi khung và chạy vòng điều khiển, và nguyên nhân của chênh lệch chưa được tách ra. Khung một tay không rẻ hơn khung hai tay (lý do có thể là bộ dò tiếp tục tìm bàn tay thứ hai còn thiếu, chưa kiểm). Nâng thu hình lên 1280×720 không làm đổi chi phí khi không có tay (27,7–31,3 ms ở cả bốn phiên); với một tay, trung bình ở C và D cao hơn A (55,5 và 52,4 so với 45,5 ms), nhưng các phiên chưa đủ so sánh được để gán cho độ phân giải.



Hình 10: (a) Thời gian suy luận điểm mốc theo phiên và số tay trong khung (hộp: tứ phân vị; râu không tính điểm ngoại lai). (b) Độ trễ từ khung camera mà bộ lọc cử chỉ chốt một cử chỉ chuyển động đến khi bánh chịu tải nhiều nhất đạt 50 % mục tiêu mới, phiên A (15 lần; nét đứt: trung vị).

Độ trễ đầu cuối (phiên A). Với 15 cử chỉ chuyển động đã chốt, trung vị thời gian từ khung camera mà bộ lọc cử chỉ chốt đến khi bánh chịu tải nhiều nhất đạt 50 % mục tiêu mới là 427 ms (321–496 ms, Hình 10b); lệnh rời laptop sau khung đó 37 ms (trung vị). Bản thân bộ lọc cần 4 khung liên tiếp, tức khoảng ba khoảng khung (120–140 ms ở các tốc độ khung này) trước khung chốt, phần này không được tính. Độ lớn mục tiêu trung vị là 0,81 tốc độ tối đa; với giới hạn gia tốc của máy chủ 2,0 s^{-1}, một nửa của nó mất khoảng 0,2 s, nên một phần lớn độ trễ là đoạn tăng tốc có chủ ý. Giá trị này dài hơn 257–260 ms ở mục 5.2, vốn tính từ giá trị cử chỉ ghi trong log điều khiển thay vì từ khung camera (muộn hơn khoảng 37 ms) và dùng các mục tiêu có độ lớn khác; 15 lần ở đây quá ít và quá đồng đều để tách các nguyên nhân. Phiên B chỉ có một lần (319 ms, mục tiêu 0,5).

Bám tốc độ và đường truyền. Với cùng quy tắc như mục 5.2, sai số trung bình là 1,3 % (thủ công) và 1,5 % (cử chỉ) tốc độ tối đa ở A, và 0,9 % ở B (chỉ cử chỉ; 12 mẫu thủ công quá ít để báo cáo), với phân vị 95 là 4,3, 5,0 và 4,2 %. Ở A, độ rộng xung chạm trần 0,95 trong 8,0 % số mẫu thủ công và 3,8 % số mẫu cử chỉ, so với 0,0 % ở B: ở tốc độ cao hơn của A bộ điều khiển hết dư địa điện áp nên không đạt được tốc độ lệnh, có thể làm méo chuyển động chéo nếu các bánh bão hòa không đều (chưa đo). Độ trễ khứ hồi UDP có trung vị 6,4 và 7,9 ms (phân vị 95 là 23,9 và 17,3 ms; lớn nhất 104 và 47 ms), phù hợp với các lần chạy trước.

Con trỏ gần mép khung. Ánh xạ ở mục 4.2 được đưa vào sau khi người vận hành báo rằng tay giao diện bị mất mỗi khi đầu ngón phải tới sát viền khung để chạm thanh tốc độ ở góc màn hình, rồi báo rằng con trỏ chạy trước đầu ngón khi đoạn kéo giãn bắt đầu đột ngột. Log chỉ cho cái nhìn gián tiếp vì không lưu vị trí đầu ngón thô: tỉ lệ khung có tay giao diện mà con trỏ cách mép màn hình dưới 2 % là 2,3 % ở A và 1,4 % ở B (ánh xạ 1:1), và 8,2 % ở C, 4,6 % ở D (kéo giãn); số đoạn tay giao diện dài hơn 0,3 s kết thúc khi con trỏ ở mép, dấu hiệu mất tay ở đó, là 1/6 ở A, 1/3 ở B, 0/15 ở C và 1/9 ở D. Người vận hành cũng cần góc nhiều hơn ở C và D vì nhiệm vụ là chạm tới đó, nên các số này chỉ cho thấy con trỏ tới được mép màn hình với ánh xạ kéo giãn, không chứng minh mất tay giảm; số đếm còn nhỏ. Độ vượt trước của con trỏ so với đầu ngón gần mép (tối đa 6 % bề rộng khung theo thiết kế) chưa được đo so với vị trí đầu ngón và dựa trên báo cáo của người vận hành.

6 Giới hạn và cách tái lập

Giới hạn. Số liệu từ một robot, một sàn nhà, một người vận hành. Chưa có khảo sát người dùng. Số đo con trỏ trên tay thật (rung, lệch khi click) chỉ từ một người và 12 lần click, chưa đo tỉ lệ bấm trúng; cơ chế neo FREEZE_BACK không cho kết quả tốt hơn trong các lần đo

này. Chưa có so sánh bộ điều khiển với PID thuần hay vòng hở. Độ trễ đầu cuối chỉ đo ở một phiên (15 lần), và tác dụng của ánh xạ gần mép lên việc mất tay chưa được đo so với vị trí đầu ngón thật. Robot không có cảm biến vật cản.

Tái lập số liệu và hình. Từ thư mục gốc dự án:

`.venv/bin/python reports/make_figures.py` (sinh hình và `reports/figures/stats.json`)
`python tests/analyze_drive_log.py docs/data/v2/drive_twohand.csv`
`.venv/bin/python reports/session2.py` (phiên thứ hai: `stats_session2.json` và hình)
`python tests/analyze_run.py logs/ai_<giờ>.csv logs/drive_<giờ>.csv`
`python tests/test_hands_logic.py` `python tests/camera_fps_test.py`